KU LEUVEN

Verslag geavanceerde computerarchitectuur

Sessie 2: Race conditions

Daan Vercammen
Klaas Meersman

ELICT: ELECFS
27 februari 2025
Master in industriële wetenschappen
Academiejaar 2024-2025

1.1 Probleemstelling en doel

Het doel van deze opdracht is om dieper in te gaan op de verschillen tussen CPU en GPU. We onderzoeken dit verschil door een maximumwaarde in een array te bepalen. Deze taak kan geparalleliseerd worden op een GPU, waarbij rekening gehouden moet worden met mogelijke race conditions. We verwachten, net als voorheen, dat de grootte van de array invloed heeft op welke methode het efficiëntst is. Daarom zullen we verschillende arraygroottes testen en de resultaten visueel weergeven.

1.2 Methodologie

Voor deze vergelijking hebben we vier verschillende algoritmes geselecteerd. Ten eerste is er 'CPU Simple', dat de array één keer doorloopt en de maximumwaarde opslaat. Als een element groter is dan de huidige maximumwaarde, wordt de maximumwaarde overschreven. Ten tweede is er 'CPU Reduction'. Dit algoritme probeert hetzelfde principe als 'GPU Reduction' na te bootsen: binnen een bepaald interval wordt de maximumwaarde telkens op de eerste positie van dat interval opgeslagen. Het interval begint met een grootte van twee en wordt telkens verdubbeld, totdat het gelijk is aan de grootte van de array. Dat we deze 'Reduction'-aanpak zowel op de CPU als op de GPU implementeren, stelt ons in staat om de verschillen tussen CPU en GPU beter te analyseren. Ten slotte hebben we 'GPU Atomic' geïmplementeerd, dat gebruikmaakt van de atomicMax-functie."

1.3 Resultaten en observaties

De resultaten (Figuur 1.1) zullen in deze sectie besproken worden. We merken hierbij enkele zaken op.

1.3.1 Maximale waarde

We kunnen er zeker van zijn dat de CPU-algoritmes de correcte maximumwaarde vinden, aangezien deze sequentieel worden uitgevoerd. Ook `atomicMAXGPU` zal een correcte waarde vinden, doordat de functie `atomicMax(array, array[tid])` ervoor zorgt dat slechts één thread tegelijkertijd toegang heeft tot het array: `atomicMax(array[0], array[tid])`(appendix A, lijn 58). We verzekeren ons ervan dat reductionMAXGPU de correcte maximumwaarde vindt door de threads na elke stap te synchroniseren met `__syncthreads()` (appendix A, lijn 69).

1.3.2 Kleine array

Bij kleine arrays presteren de CPU-algoritmes significant beter dan de GPU-algoritmes. De uitvoeringstijd van de CPU-algoritmes neemt toe naarmate de array groter wordt. De GPU-algoritmes vertonen daarentegen een constantere prestatie (zolang er genoeg threads zijn om parallell uit te voeren), maar hebben een langere uitvoeringstijd bij kleine arrays. Hun individuele cores zijn immers trager dan die van een CPU. 'CPU Simple' presteert beter dan 'CPU Reduction', omdat 'CPU Simple' de array slechts één keer doorloopt en we in de CPU geen gebruik kunnen maken van parallelisatie om de 'CPU Reduction' te versnellen.

1.3.3 Middelgrote array

Bij middelgrote arrays neemt de vertraging op de CPU zodanig toe dat de GPU-algoritmes beginnen de CPU-algoritmes in te halen. Dit komt doordat de voordelen van parallelisatie pas bij grotere arrays significant worden. Daarnaast presteert het atomicMAXGPU-algoritme iets beter dan het reductionMAXGPU-algoritme.

1.3.4 Grote array

Bij grote arrays presteren de GPU-algoritmes doorgaans beter dan de CPU-algoritmes. Dit is inherent aan het feit dat CPU-algoritmes sequentieel werken, terwijl GPU's de data parallel kunnen verwerken. De plotselinge prestatiepiek die we waarnemen bij atomicMAXGPU tussen arraygroottes 10.000 en 100.000 is echter moeilijk te verklaren. Hoewel we dit artefact consistent in meerdere benchmarkruns observeerden, ondanks het feit dat we de benchmark 80 keer per arraygrootte en algoritme uitvoerden en de mediaan van deze runs namen, blijft de oorzaak onduidelijk. Mogelijke verklaringen zijn interferentie van andere GPU-taken, of fluctuaties in de GPU-frequentie. We zien ook dat het maximale aantal cores gebruikt wordt in de GPU, waardoor ook deze seriële effecten vertoont.

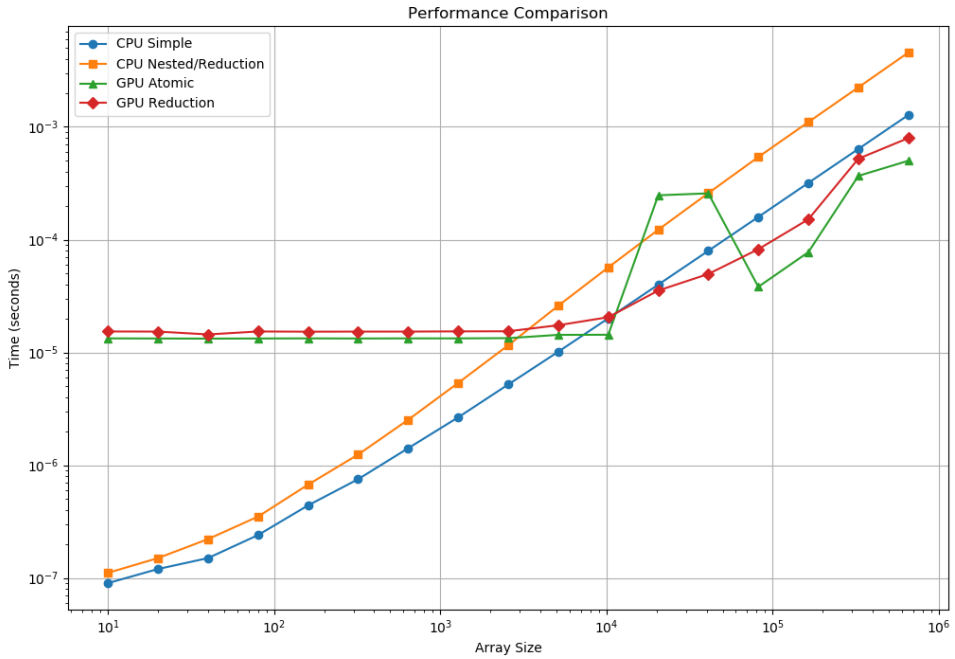
1.3.5 Array groter dan 2048

`reductionMAXGPU`(appendix A, lijn 58) is ontworpen voor één enkele block reductie. Na de reductie in shared memory, wordt het resultaat teruggeschreven naar het globale geheugen, en op de host wordt alleen deze array terug gekopiëerd. Alleen het maximum van het eerste block wordt hierdoor dus gereturned.

We kunnen dit oplossen door gebruik te maken van een reductie in twee stappen. Ten eerste voert elk blok een reductie uit op een deel van de invoerarray (deze code kan herbruikt worden). Vervolgens voert een tweede kernel-lancering een globale reductie uit op de array van blokniveau-maxima die in de eerste stap is gegenereerd om zo de algehele maximale waarde te vinden. Het is cruciaal om `cudaDeviceSynchronize()` te gebruiken tussen de kernel-lanceringen (tussen de reductie op blokniveau en de globale reductie) om te garanderen dat de vorige kernel volledig is afgerond voordat de volgende kernel begint met de uitvoering.

1.3.6 Minder threads

In dit geval moet elke thread meerdere elementen verwerken om te garanderen dat alle elementen worden overlopen.



Figuur 1.1: Uitvoertijden voor verschillende arraygroottes (logaritmische schaal)

We kunnen elke thread een deel van de array te laten doorlopen, waarbij meerdere elementen worden verwerkt. Dit kan worden bereikt door de kernel aan te passen zodat deze een lus bevat die door de array stapt met een stapgrootte die gelijk is aan het totale aantal gestarte threads.

1.3.7 2D-array

We kunnen een 2D-array aanspreken zoals onderstaande Listing.

```
1 int row = blockIdx.y * blockDim.y + threadIdx.y;
2 int col = blockIdx.x * blockDim.x + threadIdx.x;
3 int tid = row * cols + col; // Linearized index
```

1.3.8 Conclusie

De conclusie van dit labo komt grotendeels overeen met die van het vorige labo. We hebben opnieuw de uitvoeringssnelheid geanalyseerd in relatie tot de arraygrootte. We constateerden wederom dat de CPU-algoritmes sneller zijn bij kleinere arrays. Echter, dit experiment heeft ook duidelijk aangetoond dat de GPU-algoritmes superieur zijn bij grote arrays. Dit is te danken aan het feit dat de huidige bewerkingen complexer zijn dan het simpele omdraaien van een array (er is zowel een compare instructie als een schrijf instructie), waardoor de mogelijkheden tot parallelisatie op de GPU beter benut kunnen worden. Voor grote en complexe berekeningen die geparalleliseerd kunnen worden, is het dus aan te raden om gebruik te maken van een GPU. Het reductionMAXGPU-algoritme zou verder geoptimaliseerd kunnen worden.

A CODE

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <chrono>
#include <iostream>
#include <vector>
#include <random>
#include <ctime>
#include <algorithm>
#include <math.h>


int maxResultSIMPLECPU = INT_MIN;
int maxResultNESTEDCPU = INT_MIN;
int maxResultATOMICGPU = INT_MIN;
int maxResultREDUCTIONGPU = INT_MIN;

struct info {
    float kernelTime;
    float totalTime;
    int MAX;
};


int simpleMAXCPU(int *d_array, int size) {
    int MAX = INT_MIN;
    for(int i = 0;i < size; i++){
        if (d_array[i] > MAX){
            MAX = d_array[i];
        }
    }
    maxResultSIMPLECPU = MAX;
    d_array[0] = MAX;
    return MAX;
}

int nestedLoopMaxCPU(int *d_array, int size){
    int step = 1;
    int interval = 1;
    while(interval < size){
        for(int i = 0; i < size-interval; i += interval*2){
            if(d_array[i+interval]>d_array[i]){
                d_array[i] = d_array[i+interval];
            }
        }
        interval *= 2;
        step++;
    }

    maxResultNESTEDCPU = d_array[0];
```

```
    return d_array[0];
}


__global__ void atomicMAXGPU(int *d_array, int size) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;

    if (tid < size - 1) {  // Ensure we don't go out of bounds
        atomicMax(&d_array[0], d_array[tid]);
    }
}

__global__ void reductionMAXGPU(int *d_array, int size) {
    extern __shared__ int sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Load elements into shared memory
    sdata[tid] = (i < size) ? d_array[i] : INT_MIN;
    __syncthreads();

    // Perform reduction in shared memory
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = max(sdata[tid], sdata[tid + s]);
        }
        __syncthreads();
    }

    // Write result for this block to global memory
    if (tid == 0) d_array[blockIdx.x] = sdata[0];
}



info runCPUSimple(int array[], int size){
    //CPU
    //measuring
    const auto start = std::chrono::steady_clock::now();
    //measuring

    simpleMAXCPU(array, size);

    //measuring
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds{end-start};
    //measuring ;

    info info;
    info.kernelTime = elapsed_seconds.count(); // Convert to seconds
    info.totalTime = elapsed_seconds.count();   // Convert to seconds
    info.MAX = array[0];

    return info;
}

info runCPUNested(int array[], int size){
    //CPU
    //measuring
    const auto start = std::chrono::steady_clock::now();
    //measuring
```

```
111
112     nestedLoopMaxCPU(array, size);
113
114     //measuring
115     const auto end = std::chrono::steady_clock::now();
116     const std::chrono::duration<double> elapsed_seconds{end-start};
117     //measuring ;
118
119     info info;
120     info.kernelTime = elapsed_seconds.count(); // Convert to seconds
121     info.totalTime = elapsed_seconds.count();   // Convert to seconds
122     info.MAX = array[0];
123
124     return info;
125 }
126
127 info runGPUatomic(int array[], int size){
128     int *GPU_d_array;
129     cudaMalloc((void**)&GPU_d_array, size * sizeof(int));
130     int threadsPerBlock = 1024;
131     int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;
132
133     // Measure total time including data transfer
134     cudaEvent_t start_cuda_total, stop_cuda_total;
135     cudaEventCreate(&start_cuda_total);
136     cudaEventCreate(&stop_cuda_total);
137     cudaDeviceSynchronize();
138     cudaEventRecord(start_cuda_total);
139
140     // Copy data from host to device
141     cudaMemcpy(GPU_d_array, array, size * sizeof(int), cudaMemcpyHostToDevice);
142
143     // Measure kernel execution time separately
144     cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
145     cudaEventCreate(&start_cuda_kernel);
146     cudaEventCreate(&stop_cuda_kernel);
147     cudaDeviceSynchronize();
148     cudaEventRecord(start_cuda_kernel);
149
150     // Launch kernel
151     atomicMAXGPU<<<blocksPerGrid, threadsPerBlock>>>(GPU_d_array, size);
152
153     // Measure kernel execution time
154     cudaDeviceSynchronize();
155     cudaEventRecord(stop_cuda_kernel);
156     cudaEventSynchronize(stop_cuda_kernel);
157     float ms_kernel;
158     cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);
159
160     // Copy result back to host
161     cudaMemcpy(array, GPU_d_array, size * sizeof(int), cudaMemcpyDeviceToHost);
162
163     // Measure total time including data transfer
164     cudaDeviceSynchronize();
165     cudaEventRecord(stop_cuda_total);
166     cudaEventSynchronize(stop_cuda_total);
167     float ms_total;
168     cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);
169
170     // Free device memory
171     cudaFree(GPU_d_array);
```

```
    info info;
    info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
    info.totalTime = ms_total / 1000.0;   // Convert to seconds
    info.MAX = array[0];
    maxResultATOMICGPU = array[0];
    return info;
}

info runGPUReduction(int array[], int size) {
    int interval = 1;
    int step = 1;


    int *GPU_d_array;
    cudaMalloc((void**)&GPU_d_array, size * sizeof(int));


    int threadsPerBlock = 256;
    int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;

    int sharedMemSize = threadsPerBlock * sizeof(int);

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
    cudaEventCreate(&start_cuda_total);
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

    // Copy data from host to device
    cudaMemcpy(GPU_d_array, array, size * sizeof(int), cudaMemcpyHostToDevice);

    // Measure kernel execution time separately
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    // Launch kernel for reduction
    reductionMAXGPU<<<blocksPerGrid, threadsPerBlock, sharedMemSize>>>(GPU_d_array, size);


    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    // Copy result back to host (only the first element, which contains the maximum)
    cudaMemcpy(array, GPU_d_array, sizeof(int), cudaMemcpyDeviceToHost);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    // Free device memory
```

```
    cudaFree(GPU_d_array);

    info info;
    info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
    info.totalTime = ms_total / 1000.0;   // Convert to seconds
    info.MAX = array[0];
    maxResultREDUCTIONGPU = array[0];
    return info;
}

//the max value is always the size here
int* generateRandomArray(size_t size) {
    int* array = new int[size];

    // Set up random number generation
    std::mt19937 gen(static_cast<unsigned int>(std::time(nullptr)));
    std::uniform_int_distribution<> dis(0, static_cast<int>(size) - 1);
    std::uniform_int_distribution<> valueDis(0, static_cast<int>(size) - 1);

    // Fill the array with random values
    for (size_t i = 0; i < size; ++i) {
        array[i] = valueDis(gen);
    }

    // Choose a random position for the maximum value
    size_t maxPosition = dis(gen);

    // Set the maximum value at the chosen position
    array[maxPosition] = static_cast<int>(size);

    return array;
}

void printArray(int* array, size_t size) {
    for (size_t i = 0; i < size; ++i) {
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;
    printf("\n");
}


float getMedian(std::vector<float>& v) {
    size_t n = v.size() / 2;
    std::nth_element(v.begin(), v.begin()+n, v.end());
    return v[n];
}

int main() {
    const int NUM_RUNS = 80;

    for(int size = 10; size < 1000000; size *= 2) {
        std::vector<float> timesCPUSimple(NUM_RUNS);
        std::vector<float> timesCPUNested(NUM_RUNS);
        std::vector<float> timesGPUAtomic(NUM_RUNS);
        std::vector<float> timesGPUReduction(NUM_RUNS);

        for (int run = 0; run < NUM_RUNS; run++) {
            int* h_array1 = generateRandomArray(size);
            int* h_array2 = generateRandomArray(size);
            int* h_array3 = generateRandomArray(size);
```

```
            int* h_array4 = generateRandomArray(size);

            cudaDeviceSynchronize();
            timesCPUSimple[run] = runCPUSimple(h_array1, size).kernelTime;
            cudaDeviceSynchronize();
            timesCPUNested[run] = runCPUNested(h_array2, size).kernelTime;
            cudaDeviceSynchronize();
            timesGPUAtomic[run] = runGPUatomic(h_array3, size).kernelTime;
            cudaDeviceSynchronize();
            timesGPUReduction[run] = runGPUReduction(h_array4, size).kernelTime;
            cudaDeviceSynchronize();

            delete[] h_array1;
            delete[] h_array2;
            delete[] h_array3;
            delete[] h_array4;
        }

        float medianCPUSimple = getMedian(timesCPUSimple);
        float medianCPUNested = getMedian(timesCPUNested);
        float medianGPUAtomic = getMedian(timesGPUAtomic);
        float medianGPUReduction = getMedian(timesGPUReduction);

        printf("%d,%.6e,%.6e,%.6e,%.6e\n", size,
               medianCPUSimple,
               medianCPUNested,
               medianGPUAtomic,
               medianGPUReduction);
    }

    return 0;
}
```

REFERENTIELIJST

[1] *CUDA: Determining threads per block, blocks per grid*. Stack Overflow. Geraardpleegd op 17 februari 2025. 2010. URL: https://stackoverflow.com/questions/4391162/cuda-determining-threads-per-block-blocks-per-grid.

[2] NVIDIA Corporation. *CUDA C++ Programming Guide*. Geraadpleegd op 13 februari 2025. NVIDIA. verschillend. URL: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.